

Technical Architecture

Microservices on PostgreSQL · API gateway · clean code · security & observability from day one · offline-first · India-sovereign hosting.

NestJS + Go

PostgreSQL + RLS

Kong · Kafka · Temporal

OpenTelemetry + Grafana

Playwright-led testing

Architecture at a glance



Right-sized microservices

DDD bounded contexts (~10 at MVP). Avoid nano-services and the distributed monolith — split only when a context needs independent scaling or cadence.



Database-per-service

No shared database. Data is shared via APIs and events only; each service owns its PostgreSQL schema.



Events + sagas, not chains

Async Kafka events with transactional outbox; Temporal for sagas. Synchronous gRPC reserved for true reads.



Offline-first edge

Branch edge nodes keep the counter running through outages and reconcile to cloud — the core differentiator.

Clean code: hexagonal (ports & adapters) per service · CQRS read models for MIS · selective event sourcing for audit & sample lifecycle · a shared service template preloads security + observability.

Service decomposition

Platform

- **API Gateway** (Kong) + BFFs
- **Identity & Access** (Keycloak)
- **Tenant & Org**
- **Notification** (WhatsApp/SMS/email)
- **Audit & Consent** (DPDP)
- **Sync Engine** (Go)

Lab core · MVP

- **Patient (MPI)**
- **Catalog & Rate-Card**
- **Order & Workflow**
- **Result & Validation**
- **Device Gateway** (Go · HL7/ASTM)
- **Reporting · Billing**

V1 / V2

- **Referral & B2B**
- **Quality & Compliance**
- **Inventory**
- **Booking & Home-Collection**
- **MIS / Analytics**
- **Interop (ABDM) · Radiology**

Each service starts from the **Nx monorepo service template** — auth guard, OTel, RLS-aware DB, outbox + Kafka, OPA, health probes, CI pipeline — so security & observability are inherited, not retrofitted.

Decisive, locked choices

<> Backend

- **NestJS** (TypeScript) primary
- **Go** — device gateway & sync engine
- gRPC (sync) + **Kafka** (async)
- **Temporal** sagas & jobs
- Java/Spring escape hatch for V2 FHIR

🖥 Frontend

- **React + TypeScript + Vite**
- Tailwind + **shadcn/ui** + 21st.dev
- **Framer Motion** (reduced-motion)
- PWA offline counter app
- **React Native** mobile apps

🗄 Data

- **PostgreSQL 16** (db-per-service)
- **RLS** multi-tenancy + JSONB
- **Redis/Valkey** cache
- S3-compatible object store
- OpenSearch (search)

🛡 Security

- **Keycloak** OIDC + OTP/passkeys
- OPA/ABAC + RLS · **Vault**
- Linkerd mTLS · field PHI encryption
- Rate-limit + WAF/DDoS + bot defense

👁 Observability

- **OpenTelemetry** everywhere
- **Grafana LGTM** (Loki/Tempo/Mimir)
- **Sentry + PostHog**
- SLOs & error budgets

☁ Platform / infra

- **E2E Networks** (India, no hyperscaler)
- Managed K8s + **k3s** at edge
- Terraform · ArgoCD · GitHub Actions
- **Nx monorepo** + service template

Modern, user-friendly, consistent



shadcn/ui base + 21st.dev

Accessible, data-dense-friendly components; 21st.dev accelerates assembly.



Framer Motion

Micro-interactions; judicious on the counter, honors prefers-reduced-motion.



Offline counter PWA

Service worker + IndexedDB, backed by the branch edge node.



React Native apps

Patient, phlebotomist and owner apps with shared TypeScript types.

Design pipeline



Google Stitch — AI mockups



Tailwind design tokens



shadcn + 21st.dev + Framer Motion



Storybook + a11y (axe)

PostgreSQL & multi-tenancy — not MongoDB



Row-Level Security

Tenant isolation enforced in the DB — the last line of defense, not just app code.



ACID for money & PHI

Multi-row transactions and foreign keys where correctness is non-negotiable.



JSONB flexibility

FHIR bundles, device payloads & templates — document flexibility without losing relations.



Scales cleanly

Large tenants promote to a dedicated DB with no code change.

Claim propagation

JWT: tenant_id · branch_id · roles

↓ verified at gateway (Kong)

Service (RBAC + OPA/ABAC)

↓ SET app.tenant_id

Postgres RLS filters every row

Why not Mongo: no RLS equivalent, weaker multi-row ACID, and Atlas-in-India runs on excluded hyperscalers.

The core differentiator

Branch edge node



k3s + local Postgres

Counter ops run locally and never block on connectivity.



Go sync agent + device gateway

Change-log push/pull; analyzer interfacing at the edge.

Conflict strategy



Append-only favored

Orders, results & samples are low-conflict by design.



Field-level LWW

Lamport/vector clocks for mutable records.



Money = domain reconciliation

Never blind last-write-wins on billing & payments.

Hardened in testing: a Toxiproxy chaos harness injects network partitions and asserts convergence, idempotency (ULID/UUIDv7 ids) and correct money-record reconciliation.

Security & observability, built in



🛡️ Security — defense in depth

- **AuthN:** Keycloak OIDC; email/SMS/WhatsApp OTP; passkeys for staff/admin
- **AuthZ:** RBAC + OPA/ABAC + Postgres RLS
- **Secrets & transport:** Vault; Linkerd mTLS; TLS 1.3; field-level PHI encryption
- **Abuse:** rate limiting (IP/user/tenant/endpoint), WAF + DDoS (AppTrana), bot defense, OTP cost caps
- **Compliance:** DPDP consent/retention/breach; CERT-In 180-day in-India logs; hash-chained audit; VAPT

👁️ Observability — one standard

- **OpenTelemetry** traces, metrics & logs in every service
- **Grafana LGTM** — Loki, Tempo, Mimir/Prometheus (self-hosted, India)
- **Sentry** errors & release health · **PostHog** product analytics
- **RED/USE** metrics + business metrics (TAT, rejections, payout accuracy)
- SLOs & error budgets; correlation IDs across gRPC, Kafka & the edge sync boundary

India-only — no hyperscaler



A sovereignty choice. DPDP doesn't yet mandate localization, but CERT-In's 180-day in-India logs & 6-hour breach reporting are binding — and health data makes India residency the prudent default.



E2E Networks — primary

Most complete non-hyperscaler managed set: Postgres + Kafka + Valkey + K8s + S3, transparent INR pricing, MeitY + STQC.



Yotta — compliance tier

HIPAA, fuller managed-Postgres HA + PITR, Tier IV, GovCloud — for regulated & public-sector contracts.



Self-hosted on K8s

Keycloak, Temporal, Vault & the Grafana stack run on managed Kubernetes; k3s at the branch edge.

Note: independent "India-region" managed services (Aiven, Atlas, Redpanda/Temporal/Grafana Cloud) run on AWS/GCP Mumbai — excluded by the no-hyperscaler rule.

Playwright-led testing → build plan



Test pyramid

- **Unit:** Vitest · **Component:** RTL + Storybook + MSW
- **Integration:** Testcontainers + Supertest · **Contract:** Pact
- **E2E:** Playwright (web) + Maestro (mobile)
- **Load:** k6 · **Sync chaos:** Toxiproxy
- **Security CI:** Semgrep, ZAP, Trivy, gitleaks; mutation + property tests on money modules

Build plan

- **Phase 0 (3–4 wks):** monorepo + service template + CI + infra + edge skeleton
- **MVP:** epics E1–E11 as user stories
- **First slice:** the walk-in journey, end to end
- **8-sprint sequence** from foundation to MVP
- **Then:** confirm language & hosting, start Sprint S0a

ADR log records the locked decisions — authentication, multi-tenancy/RLS, offline-sync, hosting, database & backend language — each with context, decision and consequences.